



Compte rendu TP FreeRTOS



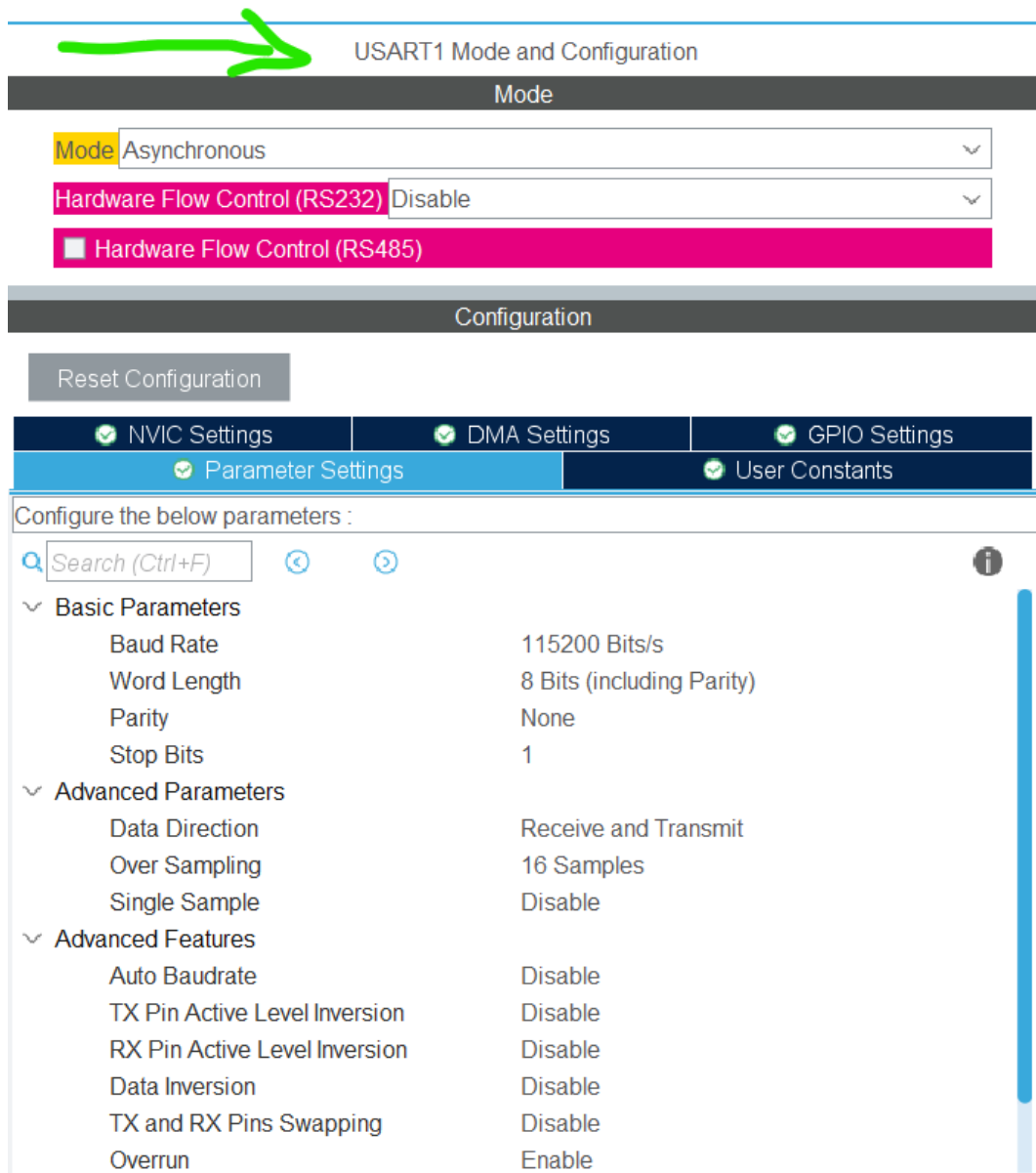
Nom du professeur : Laurent Fiack

Noms des étudiants : Farah MIR & Zakaria FOFANA

Semestre 10

2025 – 2026

0.1 Premiers pas :



1. Où se situe le fichier main.c?

Le main.c se situe dans le dossier Core/Src.

2. À quoi servent les commentaires indiquant BEGIN et END?

Ils permettent d'introduire nos codes afin de ne pas les perdre lorsqu'on régénère le code au niveau de CubeMx.

3. Quels sont les paramètres à passer à HAL_Delay et HAL_GPIO_TogglePin?

HAL_Delay :

```
HAL_Delay(500); // 500 ms est la période d'attente.
```

HAL_GPIO_TogglePin :

```
HAL_GPIO_TogglePin(GPIOI, GPIO_PIN_1);
```

GPIOI : est la définition du port I.

GPIO_PIN_1 : est la pin numéro 1 du port I.

4. Dans quel fichier les ports d'entrée/sorties sont-ils définis ?

Dans le fichier gpio.c.

5. Écrivez un programme simple permettant de faire clignoter la LED. Elle se situe sur la broche PI1.

```
void blinkLed()
{
    HAL_GPIO_TogglePin(GPIOI, GPIO_PIN_1);
    HAL_Delay(500);
}
```

6. Modifiez le programme pour que la LED s'allume lorsque le bouton USER est appuyé. Il se situe sur la broche PI11.

```
void User_Button()
{
    if (HAL_GPIO_ReadPin(GPIOI, GPIO_PIN_11) == GPIO_PIN_SET)
    {
        HAL_GPIO_WritePin(GPIOI, GPIO_PIN_1, GPIO_PIN_SET);
    }
    else
    {
        HAL_GPIO_WritePin(GPIOI, GPIO_PIN_1, GPIO_PIN_RESET);
    }
}
```

1 - FreeRTOS, tâches et sémaphores :

1.1 Tâche simple :

1. En quoi le paramètre TOTAL_HEAP_SIZE a-t-il de l'importance ?

Le paramètre **TOTAL_HEAP_SIZE** définit la quantité totale de mémoire RAM réservée au tas (heap).

Le **heap** est utilisé par FreeRTOS (création de tâches, queues, mutex, timers, stacks si allocation dynamique).

2. Créez une tâche permettant de faire changer l'état de la LED toutes les 100ms et profitez-en pour afficher du texte à chaque changement d'état. Quel est le rôle de la macro portTICK_PERIOD_MS ?

```
void led_blink_task(void* unused)
{
    //char msg[100] = "Led toggled";
    for(;;)
    {
        HAL_GPIO_TogglePin(GPIOI, GPIO_PIN_1);
        xSemaphoreTake(xPrintMutex, portMAX_DELAY);
        printf("\r\n");
        printf("Toggled\r\n");
        xSemaphoreGive(xPrintMutex);
        //printf("\r\n");
        vTaskDelay(100/portTICK_PERIOD_MS); //La LED va clignoter à chaque 100 ms
    }
}
```

```

int main(void)
{

    /* Reset of all peripherals, Initializes the Flash interface and
the Systick. */
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_USART1_UART_Init();

    xPrintMutex = xSemaphoreCreateMutex();
    configASSERT(xPrintMutex != NULL);
    // semaphore :
    xSemaphore = xSemaphoreCreateBinary();
    xPrintMutex = xSemaphoreCreateMutex();
    // tasks :

    xTaskCreate(led_blink_task, "blink", 128, NULL, 1, NULL);
    vTaskStartScheduler();
    /* USER CODE END 2 */

    /* Call init function for freertos objects (in cmsis_os2.c) */
    MX_FREERTOS_Init();

    /* Start scheduler */
    osKernelStart();

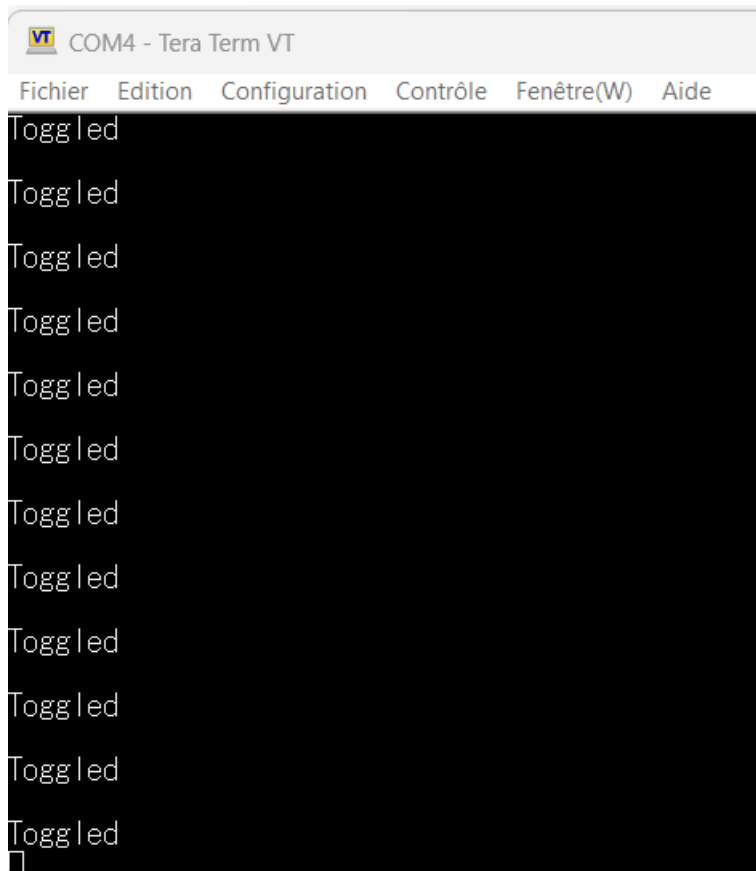
    /* We should never get here as control is now taken by the scheduler
*/

    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1)
    {

        /* USER CODE END WHILE */
        /* USER CODE BEGIN 3 */
    }
    /* USER CODE END 3 */
}

```

Sortie du terminal :



```
COM4 - Tera Term VT
Fichier  Edition  Configuration  Contrôle  Fenêtre(W)  Aide
Toggled
Toggled
Toggled
Toggled
Toggled
Toggled
Toggled
Toggled
Toggled
Toggled
Toggled
Toggled
Toggled
```

La macro `portTICK_PERIOD_MS` sert à convertir les ticks FreeRTOS en millisecondes.

1.2 Sémaphores pour la synchronisation :

3. Créez deux tâches, `taskGive` et `taskTake`, ayant deux priorités différentes. `TaskGive` donne un sémaphore toutes les 100ms. Affichez du texte avant et après avoir donné le sémaphore. `TaskTake` prend le sémaphore. Affichez du texte avant et après avoir pris le sémaphore.

```
void taskGive(void *unused)
{
    for (;;)
    {
        xSemaphoreTake(xPrintMutex, portMAX_DELAY);
        printf("\r\n");
        printf("[tak Give] Avant give\r\n");
        xSemaphoreGive(xPrintMutex);
    }
}
```

```

xSemaphoreGive(xSemaphore);

xSemaphoreTake(xPrintMutex, portMAX_DELAY);
printf("[task Give] Après give\r\n");
xSemaphoreGive(xPrintMutex);

vTaskDelay(pdMS_TO_TICKS(100));
}
}

void taskTake(void *unused)
{
    for (;;)
    {
        xSemaphoreTake(xPrintMutex, portMAX_DELAY);
        printf("[task Take] Avant take\r\n");
        xSemaphoreGive(xPrintMutex);

        xSemaphoreTake(xSemaphore, portMAX_DELAY);

        xSemaphoreTake(xPrintMutex, portMAX_DELAY);
        printf("[task Take] Après take\r\n");
        xSemaphoreGive(xPrintMutex);
    }
}

int main(void)
{
    HAL_Init();

    /* Configure the system clock */
    SystemClock_Config();
    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_USART1_UART_Init();
    /* USER CODE BEGIN 2 */

    //Mutex :

```

```

xPrintMutex = xSemaphoreCreateMutex();
configASSERT(xPrintMutex != NULL);

// semaphore :
xSemaphore = xSemaphoreCreateBinary();

xPrintMutex = xSemaphoreCreateMutex();

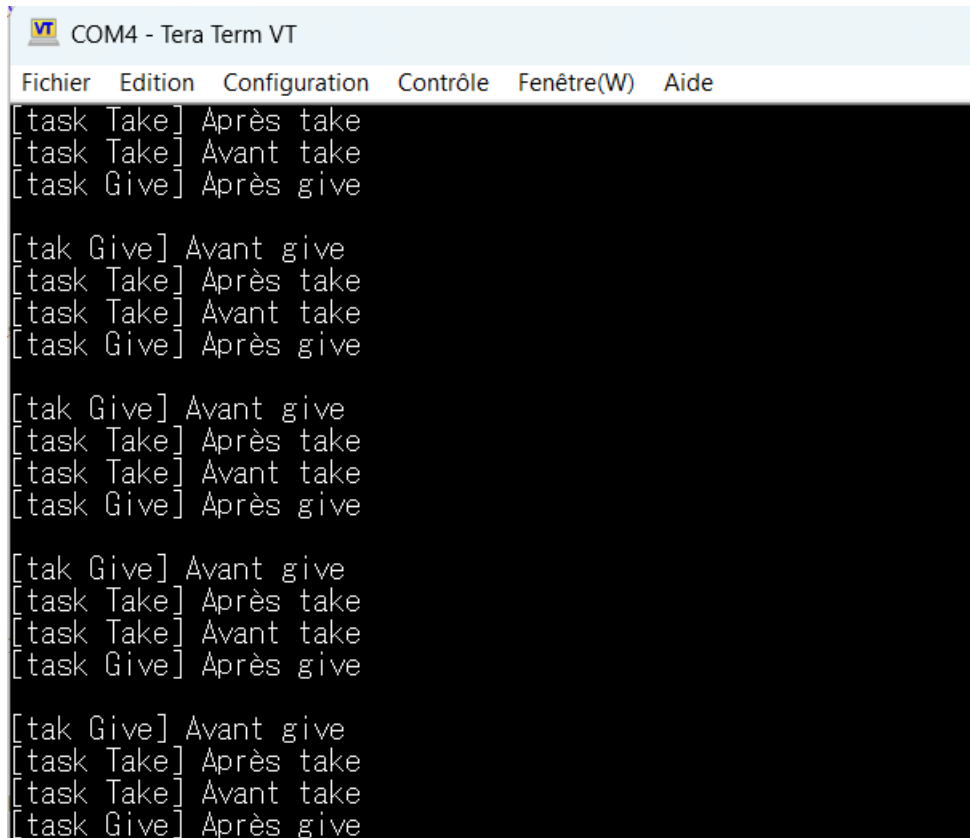
if (xSemaphore == NULL || xPrintMutex == NULL)
{
    printf("Erreur : impossible de créer le sémaphore/mutex\r\n");
    while (1);
}

xTaskCreate(taskGive, "give", 256, NULL, 2, NULL);
xTaskCreate(taskTake, "take", 256, NULL, 3, NULL);
vTaskStartScheduler();
/* USER CODE END 2 */
/* Call init function for freertos objects (in cmsis_os2.c) */
MX_FREERTOS_Init();
/* Start scheduler */
osKernelStart();
/* We should never get here as control is now taken by the scheduler
*/

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}

```


Sortie du terminal :



```
COM4 - Tera Term VT
Fichier  Edition  Configuration  Contrôle  Fenêtre(W)  Aide

[task Take] Après take
[task Take] Avant take
[task Give] Après give

[tak Give] Avant give
[task Take] Après take
[task Take] Avant take
[task Give] Après give

[tak Give] Avant give
[task Take] Après take
[task Take] Avant take
[task Give] Après give

[tak Give] Avant give
[task Take] Après take
[task Take] Avant take
[task Give] Après give

[tak Give] Avant give
[task Take] Après take
[task Take] Avant take
[task Give] Après give
```

4. Ajoutez un mécanisme de gestion d'erreur lors de l'acquisition du sémaphore. On pourra par exemple invoquer un reset software au STM32 si le sémaphore n'est pas acquis au bout d'une seconde.

```
//Fonction give gestion d'erreur
void taskGive(void *unused)
{
    for (;;)
    {
        xSemaphoreTake(xPrintMutex, portMAX_DELAY);
        printf("\r\n[task Give] Avant give (delay = %lu ms)\r\n",
delay);
        xSemaphoreGive(xPrintMutex);

        xSemaphoreGive(xSemaphore);
    }
}
```

```

        xSemaphoreTake(xPrintMutex, portMAX_DELAY);
        printf("[task Give] Après give\r\n");
        xSemaphoreGive(xPrintMutex);

        vTaskDelay(pdMS_TO_TICKS(delay));
    }
}

//Fonction take de la gestion d'erreur
void taskTake(void *unused)
{
    for (;;)
    {
        xSemaphoreTake(xPrintMutex, portMAX_DELAY);
        printf("\r\n[task Take] Avant take (attente sémaphore)\r\n");
        xSemaphoreGive(xPrintMutex);

        // Timeout de 1000 ms
        if (xSemaphoreTake(xSemaphore, pdMS_TO_TICKS(1000)) ==
pdTRUE)
        {
            xSemaphoreTake(xPrintMutex, portMAX_DELAY);
            printf("[task Take] Après take (sémaphore reçu)\r\n");
            xSemaphoreGive(xPrintMutex);
        }
        else
        {
            // ERREUR : sémaphore non reçu dans les temps
            xSemaphoreTake(xPrintMutex, portMAX_DELAY);
            printf("[task Take] ERREUR : sémaphore non reçu en 1 seconde !
RESET...\r\n");
            xSemaphoreGive(xPrintMutex);

            // Reset logiciel du STM32
            NVIC_SystemReset();
        }
    }
}

```

5. Pour valider la gestion d'erreur, ajoutez 100ms au delai de TaskGive à chaque itération.

```
//Fonction give gestion d'erreur
void taskGive(void *unused)
{
    static uint32_t delay = 100;  // commence à 100 ms

    for (;;)
    {
        xSemaphoreTake(xPrintMutex, portMAX_DELAY);
        printf("\r\n[task Give] Avant give (delay = %lu ms)\r\n",
delay);
        xSemaphoreGive(xPrintMutex);

        xSemaphoreGive(xSemaphore);

        xSemaphoreTake(xPrintMutex, portMAX_DELAY);
        printf("[task Give] Après give\r\n");
        xSemaphoreGive(xPrintMutex);

        vTaskDelay(pdMS_TO_TICKS(delay));

        delay += 100;  // on ajoute 100 ms à chaque tour
    }
}
```

Sortie du terminal :

```
COM4 - Tera Term VT
Fichier Edition Configuration Contrôle Fenêtre(W) Aide
[task Give] Avant give (delay = 800 ms)
[task Take] Après take (sémaphore reçu)

[task Take] Avant take (attente sémaphore)
[task Give] Après give

[task Give] Avant give (delay = 900 ms)
[task Take] Après take (sémaphore reçu)

[task Take] Avant take (attente sémaphore)
[task Give] Après give

[task Give] Avant give (delay = 1000 ms)
[task Take] Après take (sémaphore reçu)

[task Take] Avant take (attente sémaphore)
[task Give] Après give
[task Take] ERREUR : sémaphore non reçu en 1 seconde ! RESET... ✓

[task Take] Avant take (attente sémaphore)

[task Give] Avant give (delay = 100 ms)
[task Take] Après take (sémaphore reçu)
```

6. Changez les priorités. Expliquez les changements dans l'affichage.

Avec l'inversion des priorités nous avons un retard de 100 ms.

On a 1100ms au lieu de 1000ms.

Sortie du terminal :

```
COM4 - Tera Term VT
Fichier Edition Configuration Contrôle Fenêtre(W) Aide
[task Give] Avant give (delay = 900 ms)
[task Give] Après give
[task Take] Après take (sémaphore reçu)

[task Take] Avant take (attente sémaphore)

[task Give] Avant give (delay = 1000 ms)
[task Give] Après give
[task Take] Après take (sémaphore reçu)

[task Take] Avant take (attente sémaphore)

[task Give] Avant give (delay = 1100 ms) ✓
[task Give] Après give
[task Take] Après take (sémaphore reçu)

[task Take] Avant take (attente sémaphore)
[task Take] ERREUR : sémaphore non reçu en 1 seconde ! RESET... ✓

[task Give] Avant give (delay = 100 ms)
[task Give] Après give

[task Take] Avant take (attente sémaphore)
[task Take] Après take (sémaphore reçu)
```

1.3 Notification :

7. Modifiez le code pour obtenir le même fonctionnement en utilisant des task notifications à la place des sémaphores.

```
//Fonction take pour notifications
void taskGive(void *unused)
{
    static uint32_t delay = 100;

    for (;;)
    {
        xSemaphoreTake(xPrintMutex, portMAX_DELAY);
        printf("\r\n[Give] Avant notification (delay = %lu ms)\r\n",
delay);
        xSemaphoreGive(xPrintMutex);

        // Envoi de la notification à taskTake
        xTaskNotifyGive(taskTakeHandle);

        xSemaphoreTake(xPrintMutex, portMAX_DELAY);
        printf("[Give] Après notification\r\n");
        xSemaphoreGive(xPrintMutex);

        vTaskDelay(pdMS_TO_TICKS(delay));
        delay += 100; // pour tester l'erreur
    }
}

//Fonction take pour notifications
void taskTake(void *unused)
{
    for (;;)
    {
        xSemaphoreTake(xPrintMutex, portMAX_DELAY);
        printf("\r\n[Take] Avant attente notification\r\n");
        xSemaphoreGive(xPrintMutex);

        // Attend une notification pendant 1 seconde
        uint32_t result = ulTaskNotifyTake(pdTRUE,
pdMS_TO_TICKS(1000));
```

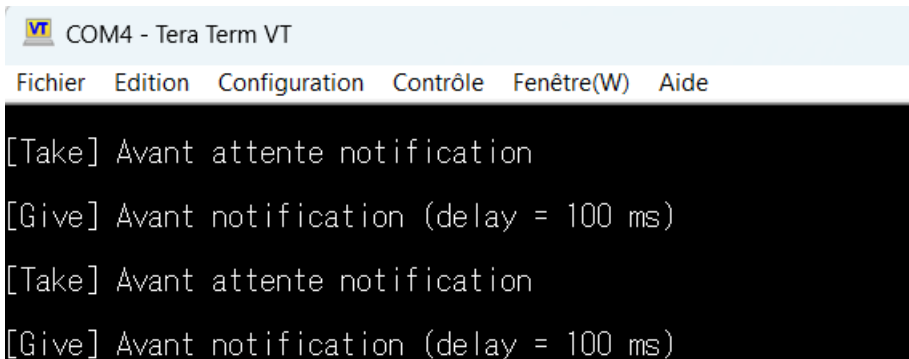
```

    if (result > 0)
    {
        xSemaphoreTake(xPrintMutex, portMAX_DELAY);
        printf("[Take] Notification reçue !\r\n");
        xSemaphoreGive(xPrintMutex);
    }
    else
    {
        xSemaphoreTake(xPrintMutex, portMAX_DELAY);
        printf("[Take] ERREUR : pas de notification en 1 seconde
! RESET...\r\n");
        xSemaphoreGive(xPrintMutex);

        NVIC_SystemReset();
    }
}

```

Sortie du terminal :



```

COM4 - Tera Term VT
Fichier  Edition  Configuration  Contrôle  Fenêtre(W)  Aide

[Take] Avant attente notification
[Give] Avant notification (delay = 100 ms)
[Take] Avant attente notification
[Give] Avant notification (delay = 100 ms)

```

1.4 Queues :

8. Modifiez TaskGive pour envoyer dans une queue la valeur du timer. Modifiez TaskTake pour réceptionner et afficher cette valeur.

```

//Fonction give pour queue
void taskGive(void *unused)
{
    uint32_t tickValue;

    for (;;)
    {
        /* Lecture du timer RTOS */
        tickValue = xTaskGetTickCount();
    }
}

```

```

xSemaphoreTake(xPrintMutex, portMAX_DELAY);
printf("\r\n[Give] Envoi tick = %lu\r\n", tickValue);
xSemaphoreGive(xPrintMutex);

/* Envoi dans la queue */
xQueueSend(xQueue, &tickValue, portMAX_DELAY);

vTaskDelay(pdMS_TO_TICKS(100));
}
}

//Fonction take pour queue
void taskTake(void *unused)
{
    uint32_t receivedTick;

    for (;;)
    {
        xSemaphoreTake(xPrintMutex, portMAX_DELAY);
        printf("[Take] En attente de données...\r\n");
        xSemaphoreGive(xPrintMutex);

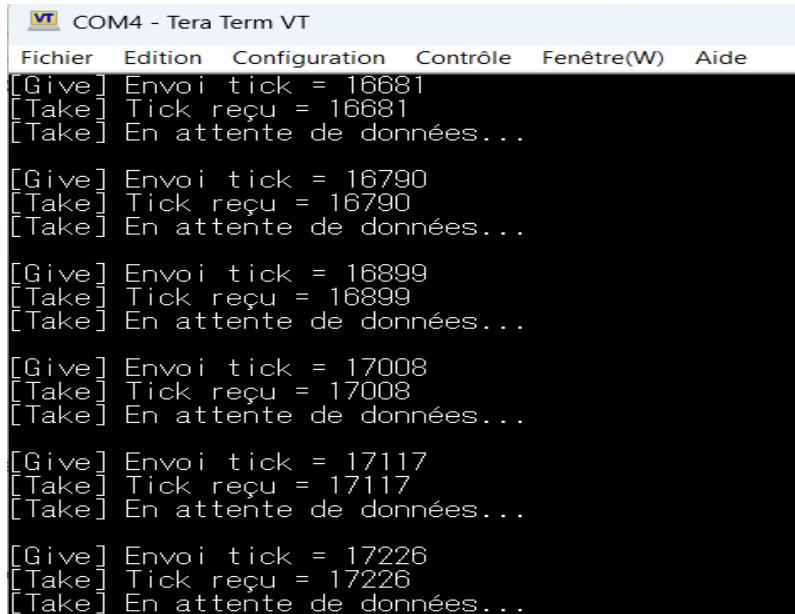
        /* Réception avec timeout de 1 seconde */
        if (xQueueReceive(xQueue,
                        &receivedTick,
                        pdMS_TO_TICKS(1000)) == pdPASS)
        {
            xSemaphoreTake(xPrintMutex, portMAX_DELAY);
            printf("[Take] Tick reçu = %lu\r\n", receivedTick);
            xSemaphoreGive(xPrintMutex);
        }
        else
        {
            /* Erreur : aucune donnée reçue */
            xSemaphoreTake(xPrintMutex, portMAX_DELAY);
            printf("[ERREUR] Timeout queue -> Reset\r\n");
            xSemaphoreGive(xPrintMutex);

            vTaskDelay(pdMS_TO_TICKS(10));
            NVIC_SystemReset();
        }
    }
}

```

```
}  
}  
}
```

Sortie du terminal :



```
COM4 - Tera Term VT  
Fichier  Edition  Configuration  Contrôle  Fenêtre(W)  Aide  
[Give] Envoi tick = 16681  
[Take] Tick reçu = 16681  
[Take] En attente de données...  
  
[Give] Envoi tick = 16790  
[Take] Tick reçu = 16790  
[Take] En attente de données...  
  
[Give] Envoi tick = 16899  
[Take] Tick reçu = 16899  
[Take] En attente de données...  
  
[Give] Envoi tick = 17008  
[Take] Tick reçu = 17008  
[Take] En attente de données...  
  
[Give] Envoi tick = 17117  
[Take] Tick reçu = 17117  
[Take] En attente de données...  
  
[Give] Envoi tick = 17226  
[Take] Tick reçu = 17226  
[Take] En attente de données...
```

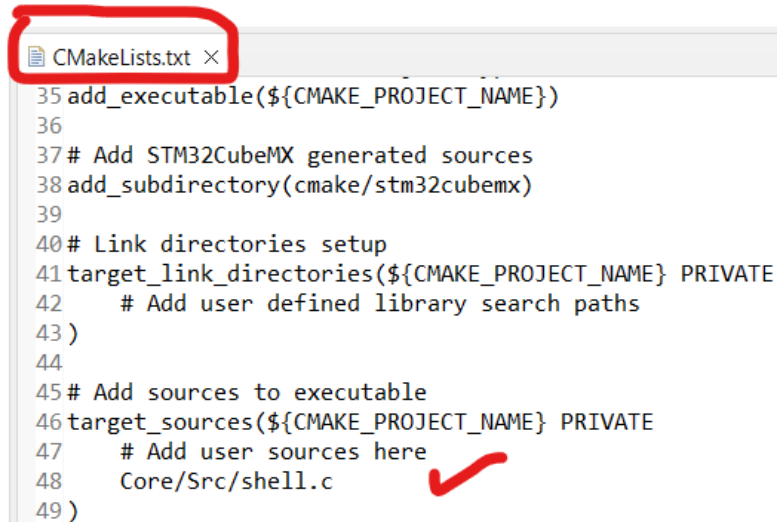
1.5 Réentrance et exclusion mutuelle :

10. Observez attentivement la sortie dans la console. Expliquez d'où vient le problème.

Sortie du terminal :

}

2 On joue avec le Shell :



```
35 add_executable(${CMAKE_PROJECT_NAME})
36
37 # Add STM32CubeMX generated sources
38 add_subdirectory(cmake/stm32cubemx)
39
40 # Link directories setup
41 target_link_directories(${CMAKE_PROJECT_NAME} PRIVATE
42     # Add user defined library search paths
43 )
44
45 # Add sources to executable
46 target_sources(${CMAKE_PROJECT_NAME} PRIVATE
47     # Add user sources here
48     Core/Src/shell.c
49 )
```

2. Que se passe-t-il si l'on ne respecte pas les priorités décrites précédemment?

Si l'on ne respecte pas les priorités d'interruptions imposées par FreeRTOS, une interruption trop prioritaire peut appeler des primitives du noyau alors que celui-ci est dans une section critique. Cela entraîne une corruption interne, provoquant des blocages, des comportements imprévisibles ou des fautes processeur.

3. Écrire une fonction `led()`, appellable depuis le shell, permettant de faire cli gnoter la LED (PI1 sur la carte). Un paramètre de cette fonction configure la periode de clignotement. Une valeur de 0 maintient la LED éteinte. Le clignotement de la LED s'effectue dans une tâche. Il faut donc trouver un moyen de faire communiquer *proprement* la fonction `led` avec la tâche de clignotement.

```

void led_task(void *arg)
{

//Suspend() et resume()


    for (;;)
    {
        if (led_period == 0)
        {
            HAL_GPIO_WritePin(GPIOI, GPIO_PIN_1, GPIO_PIN_RESET);
            vTaskDelay(10);
        }
        else
        {
            HAL_GPIO_TogglePin(GPIOI, GPIO_PIN_1);
            vTaskDelay(pdMS_TO_TICKS(led_period));
        }
    }
}

int sh_stop(int argc, char **argv)
{
    vTaskSuspend(ledTaskHandle);
    printf("LED task suspended\r\n");
    return 0;
}

int sh_start(int argc, char **argv)
{
    vTaskResume(ledTaskHandle);
    printf("LED task resumed\r\n");
    return 0;
}

```

Sortie du terminal :

```
VT COM4 - Tera Term VT
Fichier  Edition  Configuration  Contrôle  Fenêtre(W)  Aide

===== Monsieur Shell v0.2 =====
>
===== Monsieur Shell v0.2 =====
> l 200
LED period set to 200 ms
> l 500
LED period set to 500 ms
> p
LED task suspended
> r
LED task resumed
> 
```

4. Écrire une fonction `spam()`, semblable à la fonction `led()` qui affiche du texte dans la liaison série au lieu de faire clignoter les LED. On peut ajouter comme argument le message à afficher et le nombre de valeurs à afficher. Ce genre de fonction peut être utile lorsque l'on travaille avec un capteur.

```
169 void spam_task(void *arg)
170 {
171     for (;;)
172     {
173         if (spam_count > 0)
174         {
175             xSemaphoreTake(xPrintMutex, portMAX_DELAY);
176
177             printf("%s\r\n", spam_msg);
178             xSemaphoreGive(xPrintMutex);
179
180             spam_count--;
181         }
182
183         vTaskDelay(pdMS_TO_TICKS(200));
184     }
185 }
```

Sortie du terminal:

```
VT COM4 - Tera Term VT
Fichier  Edition  Configuration  Contrôle  Fenêtre(W)  Aide

===== Monsieur Shell v0.2 =====
>
===== Monsieur Shell v0.2 =====
> spam Hello 5
Spamming 'Hello' 5 times
> Hello
Hello
Hello
Hello
Hello
```

3 Debug, gestion d'erreur et statistiques :

Ce TP se réalise dans le même projet, à la suite du TP précédent. On part donc du principe que le shell est fonctionnel et utilise un mécanisme d'OS (sémaphore, queue ou notification) pour la synchronisation avec une interruption.

3.1 Gestion du tas Un certain nombre de fonctions de l'OS peuvent échouer.

Les fonctions finissant par **Create** font de l'allocation dynamique et peuvent échouer s'il n'y a plus assez de mémoire.

1. Quel est le nom de la zone réservée à l'allocation dynamique ?

La zone réservée à l'allocation dynamique s'appelle **le tas**.

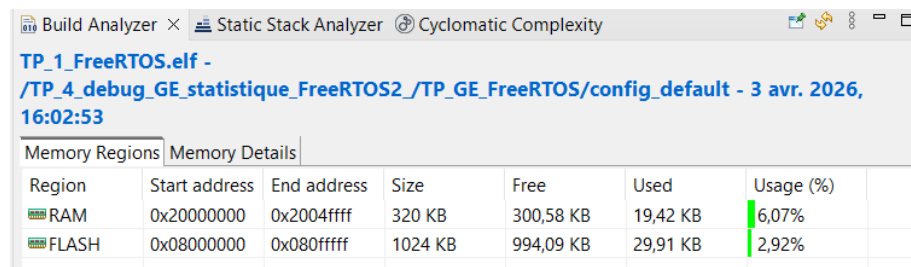
2. Est-ce géré par FreeRTOS ou la HAL?

Par FreeRTOS.

3. Si ce n'est déjà fait, ajoutez de la gestion d'erreur sur toutes les fonctions pouvant générer des erreurs. En cas d'erreur, affichez un message et appelez la fonction `Error_Handler()`;

4. Notez la mémoire RAM et Flash utilisée, comme dans l'exemple ci-dessous.

```
70 #define configTOTAL_HEAP_SIZE ((size_t)15400)
```



Region	Start address	End address	Size	Free	Used	Usage (%)
RAM	0x20000000	0x2004ffff	320 KB	300,58 KB	19,42 KB	6,07%
FLASH	0x08000000	0x080fffff	1024 KB	994,09 KB	29,91 KB	2,92%

5. Créez des tâches bidon jusqu'à avoir une erreur.

```
187 //Création des tâches bidons :
188 for (int i = 0; i < 50; i++) {
189     char name[16];
190     sprintf(name, "T%d", i);
191
192     BaseType_t ret = xTaskCreate(spam_task, name, 300, NULL, 1, NULL); //On a 256 mots donc en octet ; on a 256 x4
193
194     if (ret != pdPASS) {
195         printf("Erreur: plus de memoire pour creer la tache %d\n", i);
196         Error_Handler();
197     }
198 }
199
```

Sortie du terminal :

```

VT COM4 - Tera Term VT
Fichier  Edition  Configuration  Contrôle  Fenêtre(W)  Aide

===== Monsieur Shell v0.2 =====
Erreur: plus de memoire pour creer la tache 11

===== Monsieur Shell v0.2 =====
Erreur: plus de memoire pour creer la tache 13

```

6. Notez la nouvelle utilisation mémoire.

```

70 #define configTOTAL_HEAP_SIZE ((size_t)25400)

```

TP_1_FreeRTOS.elf - /TP_4_debug_GE_statistique_FreeRTOS2/TP_GE_FreeRTOS/config_default - 3
avr. 2026, 16:08:10

Region	Start address	End address	Size	Free	Used	Usage (%)
RAM	0x20000000	0x2004ffff	320 KB	290,81 KB	29,19 KB	9,12%
FLASH	0x08000000	0x080fffff	1024 KB	994,09 KB	29,91 KB	2,92%

7. Dans CubeMX, augmentez la taille du tas (TOTAL_HEAP_SIZE). Générez le code, compilez et testez.

Configure the below parameters :

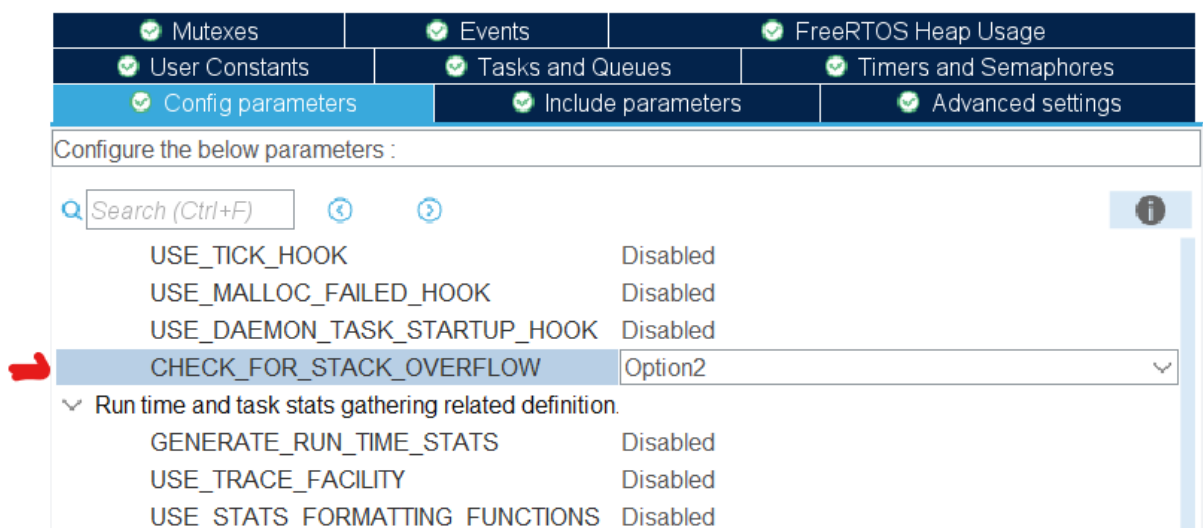
- USE_TICKLESS_IDLE: Disabled
- USE_TASK_NOTIFICATIONS: Enabled
- RECORD_STACK_HIGH_ADDRESS: Disabled
- Memory management settings**
 - Memory Allocation: Dynamic / Static
 - TOTAL_HEAP_SIZE: 30000 Bytes**
 - Memory Management scheme: heap_4
- Hook function related definitions**
 - USE_IDLE_HOOK: Disabled
 - USE_TICK_HOOK: Disabled
 - USE_MALLOC_FAILED_HOOK: Disabled
 - USE_DAEMON_TASK_STARTUP_HOOK: Disabled
 - CHECK_FOR_STACK_OVERFLOW: Disabled

8. Notez la nouvelle utilisation mémoire. Expliquez les trois relevés.

Build Analyzer × Static Stack Analyzer Cyclomatic Complexity						
TP_1_FreeRTOS.elf - /TP_4_debug_GE_statistique_FreeRTOS2/TP_GE_FreeRTOS/config_default - 3 avr. 2026, 16:35:29						
Memory Regions		Memory Details				
Region	Start address	End address	Size	Free	Used	Usage (%)
RAM	0x20000000	0x2004ffff	320 KB	286,32 KB	33,68 KB	10,52%
FLASH	0x08000000	0x080fffff	1024 KB	991,79 KB	32,21 KB	3,15%

3.2 Gestion des piles :

2. Dans CubeMX, configurez **CHECK_FOR_STACK_OVERFLOW**



3. Écriture la fonction `vApplicationStackOverflowHook`. (Rappel : C'est une fonction appelée automatiquement par FreeRTOS, vous n'avez pas à l'appeler vous-même).

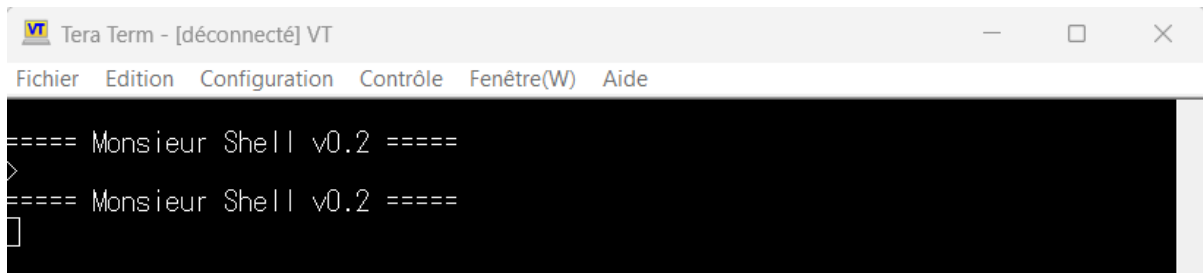
Dans le main on a augmenté la taille de la tâche :

```
203 xTaskCreate(shell_task, "shell_task", 7680, NULL, 1, NULL); // On a 256 mots donc en octet ; on a 256 x4
...
```

Et la fonction hook est :

```
282 void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName)
283 {
284     taskDISABLE_INTERRUPTS();
285     printf("Erreur : stack overflow dans la tâche %s\n", pcTaskName);
286     HAL_GPIO_WritePin(GPIOI, GPIO_PIN_1, GPIO_PIN_SET);
287     while (1) {}
288 }
289
290
291
```

Sortie du shell : Sur teraterm la tâche shell est bloquée par la boucle while de **`vApplicationStackOverflowHook`**.



4. Débrouillez-vous pour remplir la pile d'une tâche pour tester. Notez que, vu le contexte d'erreur, il ne sera peut-être pas possible de faire grand-chose dans cette fonction. Utilisez le debugger.

```
119 void task_overflow(void *arg)
120 {
121     // Grosse variable locale = explosion de pile
122     uint8_t big_buffer[500];
123
124     for (;;)
125     {
126         big_buffer[0] = 42; // juste pour éviter l'optimisation
127         vTaskDelay(10);
128     }
129 }
130
131 void boom()
132 {
133     uint8_t buffer[200];
134     boom(); // récursion infinie
135 }
136
137 void task_overflow2(void *arg)
138 {
139     boom();
140 }
```

5. Il existe d'autres hooks. Expliquez l'intérêt de chacun d'entre eux.

vApplicationMallocFailedHook() :

Intérêt

- Détecte :
 - Heap trop petit
 - Fuites mémoire

vApplicationIdleHook() :

Intérêt

- Mesure usage CPU
- Mise en veille (low power)

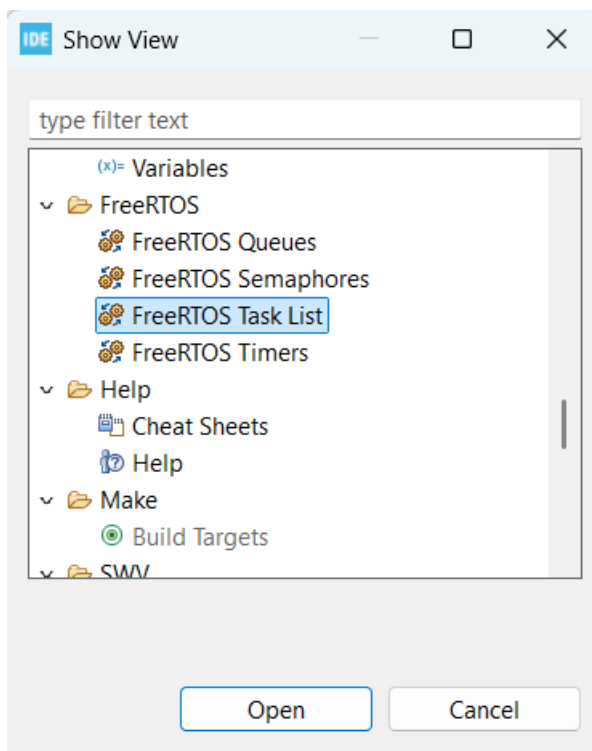
- Instrumenter l'inactivité système

vApplicationTickHook() :

Intérêt

- Timestamp
- Surveillance temps réel
- Déclenchement léger

3.3 Statistique dans l'IDE :



FreeRTOS Task List								
Name	Priority (Bas...	Start of Stack	Top of Stack	State	Event Object	Min Free St...	Run Time (%)	
⇒ IDLE	0/0	0x20000140	0x200004d4 <xldleStack+916>	RUNNING		>256	N/A	
shell_task	1/1	0x20000b28	0x20000e54 <ucHeap+1060>	BLOCKED	0x20000ab...	>256	N/A	
Spam_task	1/1	0x20000f98	0x20001324 <ucHeap+2292>	DELAYED		>256	N/A	

5. Cherchez dans CubeMX comment faire pour afficher l'utilisation de la pile. En mode debug, cliquez sur Toggle Stack Checking (dans l'onglet FreeR TOSTask List en haut à droite).

Avant l'appui du bouton :

Name	Priority (Bas...	Start of Stack	Top of Stack	State	Event Object	Stack Usage	Run Time (%)
→ IDLE	0/0	0x20000144	0x200004ec <xIdleStack+936>	RUNNING		N/A	0%
shell_task	1/1	0x20000b30	0x20000e5c <ucHeap+1064>	BLOCKED	0x20000ab...	N/A	0%
Spam_task	1/1	0x20000fa8	0x20001334 <ucHeap+2304>	DELAYED		N/A	100%

Après appui du bouton :

Name	Priority (Bas...	Start of Stack	Top of Stack	State	Event Object	Stack Usage	Run Time (%)
→ IDLE	0/0	0x20000144	0x200004ec <xIdleStack+936>	RUNNING		80B / 1028B (7,8%)	0%
shell_task	1/1	0x20000b30	0x20000e5c <ucHeap+1064>	BLOCKED	0x20000ab...	216B / 1024B (21,1%)	0%
Spam_task	1/1	0x20000fa8	0x20001334 <ucHeap+2304>	DELAYED		104B / 1024B (10,2%)	100%

6. Pour afficher le taux d'utilisation du CPU, il faut écrire les deux fonctions suivantes :

```
void configureTimerForRunTimeStats(void)
{
    MX_TIM2_Init(); // Initialise TIM2
    HAL_TIM_Base_Start(&htim2); // Démarre le timer
}
```

```
unsigned long getRunTimeCounterValue(void)
{
    return __HAL_TIM_GET_COUNTER(&htim2);
}
```

3.4 Affichage des statistiques dans le shell :

1. Écrire une fonction appellable depuis le shell pour afficher les statistiques dans le terminal.

```

232 int sh_stats(int argc, char **argv)
233 {
234     char buffer[512];
235
236     printf("=== Liste des tâches ===\r\n");
237     vTaskList(buffer);
238     printf("%s\r\n", buffer);
239
240     printf("=== Statistiques CPU ===\r\n");
241     vTaskGetRunTimeStats(buffer);
242     printf("%s\r\n", buffer);
243
244     return 0;
245 }

```

Appel dans le main :

```

320 shell_add('t', sh_stats, "Affiche les stats FreeRTOS");
321
322 xTaskCreate(shell_task, "shell_task", 256, NULL, 1, NULL); // On a 256 mots donc en octet ; on a 256 x4
323 xTaskCreate(StartTaskProcess, "TaskProcess", 256, NULL, 2, NULL);
324 xTaskCreate(spam_task, "Spam_task", 256, NULL, 1, NULL);
325

```

Sortie du shell :

```

===== Monsieur Shell v0.2 =====
>
===== Monsieur Shell v0.2 =====
> t
=== Liste des tâches ===
shell_task      X      1      26      1
IDLE            R      0     232      4
Spam_task       B      1     230      3
TaskProcess     B      2     194      2

=== Statistiques CPU ===
shell_task      90      <1%
IDLE            718235  99%
Spam_task       186     <1%
TaskProcess     22      <1%
> 

```